

Sommaire

[CEVFOLIO — Répartition du travail entre Mario et Marriette](#)

- [1. Objet et hypothèses](#)
- [2. Principe de répartition](#)
- [3. Plan détaillé de Mario](#)
- [4. Plan détaillé de Marriette](#)
- [5. Points de synchronisation et jalons](#)
- [6. Règles de collaboration](#)
- [7. Démarrage : première semaine jour par jour](#)
- [8. Dépendances et risques](#)
- [9. Ce que le chef de projet doit fournir](#)

CEVFOLIO — Répartition du travail entre Mario et Marriette

Document de référence	Cahier des charges CEVFOLIO v2.1 (les renvois « §x » pointent vers ses sections)
Périmètre	MVP : 21 semaines, du 2 novembre 2026 au 29 mars 2027 (dates indicatives)
Destinataires	Mario, Marriette
Statut	Pour démarrage

1. Objet et hypothèses

Ce document dit **qui fait quoi, dans quel ordre et avec quels livrables** pour construire le MVP. Il part de deux hypothèses à confirmer lors du lancement :

1. Mario et Marriette sont tous deux à l'aise sur **Laravel (PHP) et React (TypeScript)**.
2. Un designer UI/UX et une personne chargée de la validation métier (le chef de projet) sont disponibles. Si ce n'est pas le cas, Marriette absorbe les maquettes.

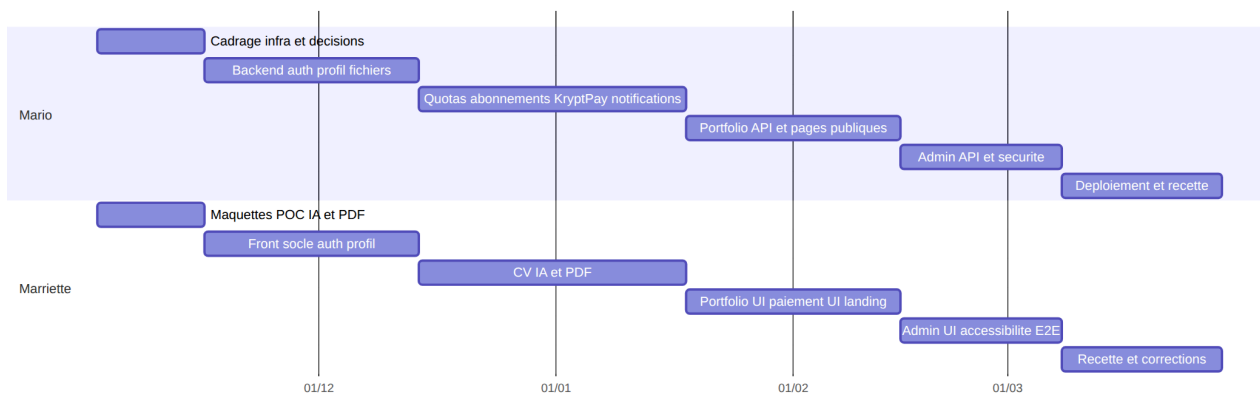
Si l'un des deux est spécialisé (par exemple uniquement backend), la répartition de la section 2 reste valable, mais les tâches d'interface sont à réaffecter.

2. Principe de répartition

Le travail est découpé **par domaine fonctionnel**, chaque personne étant propriétaire de bout en bout (API, base de données, interface) de ses modules. Cela limite les conflits de code et les attentes entre eux. Exception : la phase 1, où Mario fait le backend et Marriette le frontend du même socle (authentification et profil), pour que l'application soit utilisable dès la semaine 6.

	Mario — « Plateforme et revenus »	Marriette — « Produit : CV et IA »
Domaines propriétaires	Infrastructure, authentification, profil (API), fichiers S3, plans, quotas, abonnements, KryptPay et webhooks, notifications, portfolio (API et pages publiques), administration (API), sécurité, déploiement	Application React, design system, profil (interface), CV, templates CV, génération PDF, module IA, éditeur de portfolio, pages de tarifs et paiement (interface), administration (interface), tests E2E
Pourquoi	Ces modules sont critiques côté serveur : sécurité, argent, données	Ces modules sont le cœur de la valeur perçue par l'utilisateur et sont très liés entre eux (CV, IA, PDF)

	Mario — « Plateforme et revenus »	Marriette — « Produit : CV et IA »
Décisions préparatoires du §11 qu'il pilote	n° 1 (KryptPay), 8 (données personnelles), 7 (SEO, avec Marriette)	n° 2 (fournisseur IA), 3 (moteur PDF), 7 (SEO, avec Mario)



Le backend des abonnements (phase 4 du planning initial) est volontairement **avancé en phase 2** par Mario : la phase 2 de Marriette (CV, IA, PDF) est la plus lourde, et le quota IA a besoin des plans dès le début.

3. Plan détaillé de Mario

Phase 0 — Cadrage et infrastructure (semaines 1 à 2)

ID	Tâche	Livrable
M0.1	Créer le monorepo <code>cevfolio/</code> (<code>backend/</code> , <code>frontend/</code>), initialiser Laravel 12 (PHP 8.3). Docker Compose local : MySQL 8, Redis, MinIO (S3 local), Mailpit (e-mails)	<code>docker compose up</code> démarre tout ; README d'installation
M0.2	Intégration continue : tests (Pest ou PHPUnit), Laravel Pint, PHPStan (niveau 6), build du frontend	Pipeline vert sur chaque PR
M0.3	Migrations du socle (§7.2) : <code>users</code> (avec <code>role</code> , <code>status</code>), <code>professional_profiles</code> , les 7 tables du profil, <code>files</code> , <code>plans</code> , <code>subscriptions</code> , <code>payments</code> , <code>payment_webhooks</code> , <code>audit_logs</code> . ULID, suppression logique, Enums PHP	Migrations, modèles, factories, seeders

ID	Tâche	Livrable
M0.4	Lire la documentation officielle de KryptPay et rédiger une note d'une page : création d'un paiement, statuts, devises, abonnement récurrent ou paiement unique, signature des webhooks, environnement de test	Note de décision n° 1
M0.5	Lister les données personnelles traitées, durées de conservation, mécanisme d'effacement (§9.3)	Note de décision n° 8
M0.6	Rédiger avec Marriette le contrat OpenAPI de l'authentification et du profil	Fichier <code>openapi.yaml</code> validé par les deux

Phase 1 — Backend du socle (semaines 3 à 6)

ID	Tâche	Livrable
M1.1	Authentification Sanctum en mode SPA : inscription, connexion, déconnexion, <code>GET /me</code> , vérification d'e-mail, mot de passe oublié et réinitialisation, changement de mot de passe, <code>DELETE /me</code> (anonymisation)	Endpoints (§4.1) avec tests
M1.2	Sécurité de base : Politiques (un utilisateur n'accède qu'à ses données), compte suspendu refusé à la connexion, middleware d'administration, rate limiting sur auth	Tests d'isolation des données
M1.3	Profil : création automatique à l'inscription, <code>GET/PUT /profile</code> , CRUD complet et <code>PATCH .../reorder</code> pour expériences, formations, compétences, projets, certifications, langues et liens	API conforme au §8, FormRequests et Resources
M1.4	Service de fichiers : upload de photo et d'images, contrôle du type réel et de la taille, refus des exécutables, stockage S3, URLs signées temporaires	<code>POST/DELETE /files</code> , tests
M1.5	Journalisation, script de sauvegarde MySQL en local, seeders de démonstration	Données de test réalistes pour Marriette

Phase 2 — Quotas, abonnements, paiement (semaines 7 à 11)

ID	Tâche	Livrable
M2.1	Plans et abonnements : seeders Free et Pro (valeurs provisoires), règle « un seul abonnement actif », Free implicite (\$5.3)	SubscriptionService
M2.2	QuotaService (livré en premier, avant le jour 3 de la semaine 7) : canUse(), consume(), remaining() pour l'IA, le nombre de CV et de portfolios	Interface figée, utilisée par Marriette
M2.3	PaymentService et KryptPayGateway, implémentés d'après la documentation officielle, en environnement de test	Paiement de test réussi
M2.4	Endpoints GET /plans, POST /subscriptions, GET /subscription, POST /subscription/cancel, GET /payments	Contrat OpenAPI à jour
M2.5	Webhook POST /webhooks/kryptpay : vérification de signature, idempotence via payment_webhooks, mise à jour de Payment puis de Subscription	Tests : webhook rejoué, signature invalide, ordre inversé
M2.6	Notifications : e-mails de vérification, de réinitialisation, de paiement ; Scheduler pour SUBSCRIPTION_EXPIRING et le retour automatique au plan Free ; préférences de notification	Notifications et tâches planifiées testées
M2.7	Service d'audit (audit_logs) appelé par les actions sensibles	Journal consultable en base

Phase 3 — Portfolio (semaines 12 à 15)

ID	Tâche	Livrable
M3.1	Migrations et modèles : portfolio_templates, portfolios, portfolio_sections, custom_domains (table seule, fonctionnalité V2)	Modèle en base

ID	Tâche	Livrable
M3.2	API portfolios : CRUD, génération des sections à partir du profil via ProfileSnapshotService , publish et unpublish , slug unique, contrôle du quota	Endpoints du §8
M3.3	Pages publiques /@{slug} rendues côté serveur (Blade) avec balises meta et Open Graph ; 404 si non publié	Page publique visible et partageable
M3.4	Implémentation des 2 templates de portfolio en Blade/CSS, d'après les maquettes	2 templates utilisables

Phase 4 — Administration et sécurité (semaines 16 à 18)

ID	Tâche	Livrable
M4.1	API d'administration : utilisateurs (suspendre, réactiver), abonnements, paiements, templates, prompts IA (édition et versions), consommation IA	/admin/* testé
M4.2	Revue de sécurité : en-têtes HTTP, CORS, rate limiting, upload, analyse des dépendances, secrets uniquement côté serveur	Rapport et correctifs

Phase 5 — Mise en production (semaines 19 à 21)

ID	Tâche	Livrable
M5.1	Déploiement : serveurs, HTTPS, workers (Supervisor), cron du Scheduler, bucket S3 et ses règles d'accès	Environnement de production
M5.2	Sauvegardes MySQL chiffrées et test de restauration	Procédure validée
M5.3	Supervision et alertes : files d'attente, erreurs, webhooks en échec	Tableau de bord d'alertes
M5.4	Corrections de la recette backend, documentation technique et runbook	Dossier d'exploitation

4. Plan détaillé de Marriette

Phase 0 — Maquettes et preuves de concept (semaines 1 à 2)

ID	Tâche	Livrable
R0.1	Avec le designer : charte, tokens Tailwind (couleurs, typographies, espacements), maquettes des écrans clés (profil, éditeur de CV, éditeur de portfolio, tarifs)	Maquettes validées
R0.2	POC IA : comparer 2 fournisseurs ou modèles, écrire les 3 prompts MVP (résumé, expérience, projet), imposer une sortie JSON validée par schéma, mesurer coût et taux de JSON invalide	Note de décision n° 2 avec coût par requête
R0.3	POC PDF : un template HTML/CSS rendu par Chromium headless (ex. Browsershot) et comparé à l'aperçu à l'écran ; évaluer une alternative	Note de décision n° 3
R0.4	Trancher avec Mario le rendu des pages publiques (recommandation : Blade côté serveur)	Note de décision n° 7
R0.5	Rédiger avec Mario le contrat OpenAPI authentification et profil	<code>openapi.yaml</code>

Phase 1 — Frontend du socle (semaines 3 à 6)

ID	Tâche	Livrable
R1.1	Socle React : Vite, TypeScript strict, Tailwind, React Router, Axios (gestion du cookie CSRF Sanctum), TanStack Query, routes protégées, gestion globale des erreurs	Application qui démarre et appelle l'API
R1.2	Design system : boutons, champs, sélecteurs, modales, tableaux, toasts, états de chargement	Bibliothèque de composants
R1.3	Écrans d'authentification : inscription, connexion, vérification d'e-mail, mot de passe oublié et réinitialisation	Parcours complet
R1.4	Écrans du profil : formulaires (React Hook Form + Zod) pour identité, résumé, et les 7 entités ; réordonnement par glisser-déposer ; envoi de photo	Profil complet saisissable

ID	Tâche	Livrable
R1.5	Tests de composants (Vitest et Testing Library) sur les formulaires ; structure prête pour la traduction (interface en français)	Tests verts en CI

Phase 2 — CV, IA et PDF (semaines 7 à 11) — chemin critique

ID	Tâche	Livrable
R2.1	ProfileSnapshotService (livré avant le jour 3 de la semaine 7) : copie les éléments choisis du profil dans les sections d'un CV, et resynchronisation avec confirmation (§5.1)	Interface figée, réutilisée par Mario en phase 3
R2.2	Migrations et modèles <code>cv_templates</code> , <code>cvs</code> , <code>cv_sections</code> ; API CV : <code>CRUD</code> , <code>duplicate</code> , <code>generate</code> , <code>export</code> , <code>download</code> ; Politiques ; contrôle du quota via <code>QuotaService</code>	Endpoints du §8
R2.3	3 templates de CV en HTML/CSS (Minimal, Modern, Corporate) pilotés par <code>settings</code> : couleurs, police, taille, marges, photo, sections et ordre	3 templates fidèles au rendu PDF
R2.4	Éditeur de CV : choix du template, sélection des éléments du profil, personnalisation, aperçu en temps réel	Écran complet
R2.5	Module IA backend : abstraction du fournisseur, <code>PromptTemplate</code> (avec seeders), <code>AIRequest</code> , <code>jobGenerateAIContent</code> , validation du JSON, une nouvelle tentative puis échec sans consommer de quota, endpoints <code>/ai/summary</code> , <code>/ai/experience</code> , <code>/ai/project</code> , <code>/ai/skills</code> , <code>GET /ai/requests/{id}</code> , <code>GET /ai/quota</code>	Endpoints et tests avec fournisseur simulé
R2.6	Interface IA : boutons « Générer » et « Améliorer » dans le profil et le CV, suivi de l'état (polling), panneau « Accepter, modifier, rejeter », affichage du quota restant	Parcours IA complet

ID	Tâche	Livrable
R2.7	Job GeneratePDF : rendu, stockage S3, generation_status , notification PDF_READY , téléchargement via URL signée	PDF identique à l'aperçu

Phase 3 — Portfolio, paiement, landing (semaines 12 à 15)

ID	Tâche	Livrable
R3.1	Éditeur de portfolio : choix du template, sections activables, réordonnables et personnalisables, aperçu, publication, copie de l'URL publique	Écran complet, branché sur l'API de Mario
R3.2	Interface d'abonnement : page des tarifs, comparaison Free/Pro, démarrage du paiement selon le flux KryptPay, page de statut du paiement, gestion et annulation, affichage des limites et du quota IA	Parcours de paiement de test
R3.3	Landing page publique : proposition de valeur, tarifs, modèles (avec balises SEO)	Page en ligne en recette
R3.4	Centre de notifications et préférences	Écrans
R3.5	Durcissement du CV et de l'IA : erreurs, délais d'attente, reprise après échec	Cas d'erreur gérés

Phase 4 — Administration, qualité (semaines 16 à 18)

ID	Tâche	Livrable
R4.1	Interface d'administration : utilisateurs, abonnements, paiements, édition des prompts IA, consommation IA, templates	Back-office utilisable
R4.2	Accessibilité (objectif WCAG AA), responsive mobile, performance (Lighthouse)	Rapport et correctifs
R4.3	Tests E2E (Playwright) : inscription → profil → CV → PDF ; publication d'un portfolio ; paiement de test	Suite E2E en CI

Phase 5 — Recette (semaines 19 à 21)

ID	Tâche	Livrable
R5.1	Dérouler les critères d'acceptation du §15, corriger les anomalies frontend	Grille de recette remplie
R5.2	Vérifier le rendu SEO et Open Graph des pages publiques	Contrôle documenté
R5.3	Aide en ligne et guide utilisateur court	Guide publié

5. Points de synchronisation et jalons

Jalon	Échéance	Contenu attendu	Responsables
J0	Fin de la semaine 2	Décisions n° 1, 2, 3, 7 et 8 consignées ; environnement Docker partagé ; OpenAPI auth et profil validé	Les deux
J1	Fin de la semaine 6	Inscription, connexion et profil complet fonctionnels de bout en bout	Mario (API), Marriette (interface)
Contrat 1	Semaine 7, avant le jour 3	QuotaService (Mario) et ProfileSnapshotService (Marriette) : signatures PHP figées et documentées	Chacun pour son service
J2	Fin de la semaine 11	CV complet : création, IA, PDF. Paiement de test et webhook validés	Marriette, Mario
J3	Fin de la semaine 15	Portfolio publié sur une URL publique ; paiement de bout en bout	Les deux
J4	Fin de la semaine 18	Administration et revue de sécurité terminées ; gel des fonctionnalités	Les deux
J5	Fin de la semaine 21	Recette signée par le chef de projet ; mise en production	Les deux

Signatures proposées pour les deux services partagés :

QuotaService

```
canUse(User $user, QuotaType $type): bool
```

```
consume(User $user, QuotaType $type, int $amount = 1): void
```

```
remaining(User $user, QuotaType $type): int // QuotaType : AI, CV, PORTFOLIO
```

ProfileSnapshotService

```
snapshot(ProfessionalProfile $profile, array $selection): array // contenu des sections
resync(Model $target, array $selection): array // CV ou Portfolio
```

6. Règles de collaboration

Sujet	Règle
Dépôt et branches	Monorepo. <code>main</code> (protégée) et <code>develop</code> . Branches <code>feature/{ID}-{description}</code> . Aucun commit direct sur <code>main</code> ni <code>develop</code>
Revue de code	Toute <i>pull request</i> est relue par l'autre personne sous 24 h ouvrées . Une PR fait idéalement moins de 400 lignes
Commits	Format conventionnel (<code>feat:</code> , <code>fix:</code> , <code>test:</code> , <code>docs:</code> , <code>chore:</code>)
Définition de « terminé »	Code relu, tests écrits et verts, Pint, PHPStan et ESLint sans erreur, contrat OpenAPI à jour, aucune donnée sensible dans les journaux
Contrat d'API	L'API est décrite avant d'être codée (<code>openapi.yaml</code>). Toute modification passe par une PR étiquetée <code>api-contract</code> , approuvée par les deux
Base de données	Chaque personne crée les migrations de ses tables. Une migration fusionnée n'est jamais modifiée : on en ajoute une nouvelle
Secrets	Variables d'environnement serveur uniquement. Aucune clé (IA, KryptPay, S3) dans le frontend ni dans Git
Tickets	Un ticket par tâche des tableaux ci-dessus (ID repris dans le titre), découpé en morceaux de 2 jours maximum
Rituels	Message quotidien écrit (fait, prévu, blocages), point de 30 minutes deux fois par semaine, démonstration le vendredi, rétrospective mensuelle
Blocage	Toute attente sur l'autre dépassant 1 jour est signalée le jour même au chef de projet

7. Démarrage : première semaine jour par jour

Jour	Mario	Marriette
1	Réunion de lancement (1 h, avec le chef de projet) : valider ce document, choisir l'outil de tickets et l'hébergeur Git. Créer le monorepo et le Docker Compose (M0.1)	Lire le cahier des charges en entier et noter les questions. Initialiser le projet Vite, TypeScript et Tailwind
2	CI, Pint, PHPStan, Pest (M0.2). Début des migrations <code>users</code> et <code>professional_profiles</code>	Tokens du design system et premières maquettes avec le designer (R0.1). Préparer le POC IA
3	Migrations des entités du profil, Enums, factories, seeders (M0.3). Commencer la lecture de KryptPay	POC IA : 3 prompts, schéma JSON, tests sur 2 fournisseurs (R0.2)
4	Rédaction de la note KryptPay (M0.4). Migrations de facturation	POC PDF avec un template simple (R0.3)
5	Ensemble : rédaction et revue de <code>openapi.yaml</code> (M0.6, R0.5), démonstration des POC, consigner les décisions, créer les tickets de la phase 1	

8. Dépendances et risques

Risque ou dépendance	Impact	Parade
Documentation KryptPay incomplète ou flux récurrent non pris en charge	Retarde le paiement	Mario la traite dès la semaine 1 ; prévoir en dernier recours des renouvellements manuels
La phase 2 de Marriette (CV, IA, PDF) est la plus chargée	Retard sur le chemin critique	Si retard à la fin de la semaine 9, Mario reprend R2.7 (job PDF) et décale M2.6 et M2.7
Marriette dépend de <code>QuotaService</code> et Mario de <code>ProfileSnapshotService</code>	Blocage croisé	Signatures figées en semaine 7 (contrat 1), implémentations simulées en attendant
Fidélité entre l'aperçu et le PDF	Rework des templates	POC dès la semaine 1 ; un seul moteur de rendu
Coût et fiabilité de l'IA	Quotas mal calibrés	Mesures du POC, plafonds par utilisateur, suivi dans <code>ai_requests</code>
Valeurs non décidées (prix, quotas, export PDF en Free)	Blocage de l'interface tarifs	Valeurs provisoires dans <code>plans</code> , modifiables sans déploiement

9. Ce que le chef de projet doit fournir

Les noms et coordonnées du designer, du chef de projet et du testeur ; les comptes et clés de test (fournisseur IA, KryptPay sandbox, S3) ; l'accès au dépôt Git et à l'outil de tickets ; et, avant la fin de la semaine 2, la décision sur les prix, les quotas et la limite d'export PDF en Free.